

TUTORIAL 02

MLflow

Experiment tracking, the model registry, and versioning

DDM501 — AI in DevOps, DataOps, MLOps

1. Purposes

Tutorial 01 could not	MLflow calls it	You meet it in
Tell which run produced a file	run id	step 1
Add a column without breaking old rows	parameters	step 1
Keep a plot with the run that made it	artifacts	step 1
Compare forty runs without reading a CSV	the tracking UI	step 2
Record the spread, not just the mean	metrics	step 2
Promote a model without copying files	registry + alias	step 3
Say which library pickled a model	flavour metadata	step 4

2. Setting up

MLflow has two halves: a library you import, and a server that stores what the library sends. For this tutorial the server is a local process backed by a SQLite file. If you want to run it in virtual environment:

```
python3.11 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --artifacts-destination ./mlartifacts \
  --host 127.0.0.1 --port 5000
```

Leave that terminal running and open <http://127.0.0.1:5000> in a browser.

Or just install Docker and use “*docker compose*”

2.1 Why SQLite

Most MLflow examples online use `./mlruns` directory, which needs no server at all. It works for tracking and then fails to register a model: the registry needs a database.

SQLite is a database in a single file. Lab 2 uses PostgreSQL in a container.

2.2 The two lines that decide where your runs go

```
mlflow.set_tracking_uri("http://127.0.0.1:5000")  # WHERE runs are stored
mlflow.set_experiment("breast-cancer-diagnosis")  # WHICH drawer inside it
```

MLflow silently falls back to a local `./mlruns` folder that the UI you have open is not reading.

2.3 Running the server in a container instead

Optional, and ussing Docker:

```
docker compose up -d --build
docker compose ps          # wait for STATUS = healthy, about 40 s
# then open http://127.0.0.1:15000

docker compose run --rm runner python scripts/step1_track.py
docker compose run --rm runner python scripts/step2_compare.py
docker compose run --rm runner python scripts/step3_register.py
docker compose run --rm runner python scripts/step4_load_and_verify.py
```

The port is 15000 and not 5000.

```
export MLFLOW_TRACKING_URI=http://127.0.0.1:15000
python scripts/step1_track.py
```

A tracking server is an HTTP service; the client does not care where it lives.

On Linux, copy `.env.example` to `.env` first and put the number `id -u` prints into it, so the container writes to `mlflow-data/` as you rather than as root.

3. Step 1 — tracking

```
export PYTHONPATH=scripts
python scripts/step1_track.py    # about 80 seconds
```

12 configurations across three model families, each one logged as a run..

Kind	What goes in it	Rule
Parameter	family, C, max_depth, seeds, the CV scheme	Inputs. Strings. Written once and never updated — that is what makes a run a record rather than a variable.
Metric	cv_roc_auc_mean, cv_roc_auc_std, test recall	Outputs. Numbers. Can be logged repeatedly, e.g. once per epoch.
Tag	dataset name, library version, git branch	Free-form labels you search and filter by later.
Artifact	confusion matrix PNG, the model itself	Files. Anything you would otherwise leave on disk and lose.

The whole of one run is a block:

```
with mlflow.start_run(run_name="svc-10.0-0.01"):
    scores = cross_val_score(model, X_tr, y_tr, cv=cv, scoring="roc_auc")
    model.fit(X_tr, y_tr)

    mlflow.log_params({"family": "svc", "C": 10.0, "gamma": 0.01})
```

```
mlflow.log_metrics({"cv_roc_auc_mean": scores.mean(),
                  "cv_roc_auc_std": scores.std()})
mlflow.set_tags({"dataset": "wdbc-569", "sklearn": sklearn.__version__})
mlflow.log_artifact("artifacts/confusion_matrix.png")
mlflow.sklearn.log_model(model, artifact_path="model")
```

Note `cv_roc_auc_std`. In Tutorial 01 adding that column to the CSV meant editing the header, rewriting the writer, and leaving every existing row blank.

4. Step 2 — comparison

```
python scripts/step2_compare.py
```

It reads the server back and answers, in seconds, the questions Tutorial 01 could not answer

```
12 runs on the server.

      run family cv_mean cv_std recall_mal
svc-10.0-0.01   svc  0.99611 0.00539    0.9762
logreg-1.0     logreg 0.99525 0.00534    0.9762
logreg-0.1     logreg 0.99525 0.00566    0.9524
svc-1.0-scale   svc  0.99501 0.00530    0.9762
svc-1.0-0.01   svc  0.99487 0.00623    0.9762
svc-10.0-scale  svc  0.99370 0.00647    0.9762
rf-400-None     rf   0.98951 0.00958    0.9286
rf-400-8        rf   0.98947 0.00960    0.9286
rf-400-3        rf   0.98937 0.00754    0.9286
rf-100-None     rf   0.98842 0.01002    0.9286
```

4.1 Mean ranking results

Tutorial 01 showed that on 569 rows a ranking of similar configurations is noise.

```
within each family, best minus worst, against the family's own spread:
logreg  n= 2  spread 0.00000  mean std 0.00550  -> noise
rf      n= 6  spread 0.00117  mean std 0.00919  -> noise
svc     n= 4  spread 0.00241  mean std 0.00585  -> noise

gap over the 10th run    0.00769
std of the winner        0.00539
-> the ranking is outside the noise
```

Read those two blocks together, because they say different things. Choosing between an SVM and a random forest is a real effect on this data.

4.2 The UI

Keep the UI open at <http://127.0.0.1:5000> and work through these:

1. Select all runs, press Compare, open the Parallel Coordinates tab. Drag `cv_roc_auc_mean`. Which hyper-parameter actually moves the score?
2. Sort the table by `test_recall_malignant` instead of AUC. Does the order change? Which metric matches what a missed cancer costs?

3. Open any run and go to the Artifacts tab. The confusion matrix is attached to the run that produced it, not sitting in a folder called `plots_final_v2`.
4. Add the `cv_roc_auc_std` column to the table view. Ten runs, one column, no CSV surgery.

5. Step 3 — the registry

```
python scripts/step3_register.py
```

Tracking answers "what did I try?". The registry answers "what is deployed, and what do I roll back to?"

An experiment holds hundreds of runs; a registered model holds the

Concept	What it is
Registered model	A name, such as <code>breast-cancer-classifier</code> . Not a file.
Version	An append-only number under that name. Registering never overwrites.
Alias	A movable pointer to one version. Serving code names the alias, never a number.

The script registers two runs, tags them, then promotes the challenger:

```
registered version 1: svc-10.0-0.01
registered version 2: logreg-1.0

version  aliases          description
    1  champion          best cv_roc_auc_mean
    2  challenger        runner-up, different family, simpler

Now promote the challenger. Watch what does and does not change.

version  aliases          description
    1  -                  best cv_roc_auc_mean
    2  champion          runner-up, different family, simpler
```

Both versions are still there.

Aliases, not stages

MLflow 2.x replaced the old Staging / Production stages with aliases, and Lab 2 uses aliases. Any tutorial you find online calling `transition_model_version_stage` is written for an older version. Learn `set_registered_model_alias`.

6. Step 4 — load it back and prove it

```
python scripts/step4_load_and_verify.py
```

A registry is only useful if something downstream can pick a model up knowing nothing but a name.

```
loaded models:/breast-cancer-classifier@champion

version      2
run_id       f07a193571454ad7b22e8361258dd8d4
run_name     logreg-1.0
params       {"C": "1.0", "cv": "RepeatedStratifiedKFold(5x3, seed=0)",
              "family": "logreg", "split_seed": "42"}
logged cv    0.99525 +/- 0.00534

pickled by sklearn 1.6.0
running      sklearn 1.6.0
match -- safe to load

test_roc_auc logged at training time 0.995370
test_roc_auc recomputed just now    0.995370
identical
```

Only a model name and an alias reproduced a number from a training run it did not perform, to six decimal places, and it walked back from the alias to the version to the run to the exact parameters

7. What MLflow does not do

Worth being clear about, because the UI is persuasive.

- It does not make your model better. Step 2's conclusion — that the hyper-parameter tuning was noise — was true in Tutorial 01 too. MLflow only made it cheap to see.
- It does not run anything on a schedule. Every run in this tutorial happened because you typed a command. Tutorial 03 covers that.
- It does not watch a deployed model. A registered model that has been serving badly for three weeks looks identical in the UI to one deployed this morning. Tutorial 04 covers that.
- It does not record what it was not told. The seed, the CV scheme and the library version are in your runs because the script logged them. Nothing detects an unlogged parameter.